

Reviewer Pack — Minimal Order of Brauer Groups and Communication Complexity ...

Entry d7cf4659c191 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 14:33:12 UTC

Minimal Order of Brauer Groups and Communication Complexity Rank Variance

Entry ID: d7cf4659c191 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 14:33:12 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Brauer Group Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every Boolean function f with communication complexity rank r_f , the minimal order of its associated Brauer group $B(f)$ is upper bounded by a polynomial in $\log(r_f)$, i.e., $|B(f)| \leq p(\log(r_f))$ for some polynomial $p(n)$.

Rationale (proposer's reasoning):

The Brauer group encodes the non-trivial linear obstructions to the representation theory of a field, while communication complexity rank measures the minimum number of bits required to communicate information between two parties. This conjecture proposes that the complexity of these representations is bounded by the complexity of the communication process.

Taxonomy category: BrauerGroup (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4932783eb32e7958

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean minimal order of Brauer groups for all generated Boolean functions with communication complexity rank r_f is less than or equal to a polynomial in $\log(r_f)$, measured across 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Brauer Group Theory' AND 'communication complexity rank' AND 'matrix rank' - 'minimal order of Brauer groups' AND 'Boolean functions' AND 'polynomial bound on $\log(r_f)$ ' - polylogarithmic bound 'Brauer group' 'communication complexity'

Top relevant hits considered: - [<http://arxiv.org/abs/1403.8106v1>] Recent advances on the log-rank conjecture in communication complexity - [<http://arxiv.org/abs/1908.06409v1>] Schur multipliers of special p-groups of rank 2 - [<http://arxiv.org/abs/1102.2932v2>] Applications of Monotone Rank to Complexity Theory - [<http://arxiv.org/abs/2406.18700v4>] Structure of sparse Boolean functions over Abelian groups, and its application to testing - [<http://arxiv.org/abs/1911.04516v1>] Boolean lattices in finite alternating and symmetric groups - [<http://arxiv.org/abs/1110.2956v2>] Brauer spaces for commutative rings and structured ring spectra - [<http://arxiv.org/abs/1110.6618v2>] Brauer relations in finite groups II - quasi-elementary groups of order p^{aq} - [<http://arxiv.org/abs/2309.16111v2>] The relational complexity of linear groups acting on subspaces

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def communication_complexity_rank(f):
        # Placeholder for actual computation
        return len(f)

    def construct_matrix_representation(f, n):
        # Placeholder for actual computation
        return [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

    def gaussian_elimination(matrix):
        rows = len(matrix)
        cols = len(matrix[0])
        for i in range(rows):
            max_row = i
            for j in range(i+1, rows):
                if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                    max_row = j
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            factor = 1 / matrix[i][i]
```

```

        for j in range(cols):
            matrix[i][j] *= factor
        for j in range(rows):
            if j != i:
                factor = matrix[j][i]
                for k in range(cols):
                    matrix[j][k] -= factor * matrix[i][k]
    return matrix

def compute_brauer_group_order(matrix):
    # Placeholder for actual computation
    rank = 0
    for row in matrix:
        if any(row):
            rank += 1
    return rank ** 2

n_max = 40
instances_tested = 30
total_metric_value = 0.0

for _ in range(instances_tested):
    n = random.randint(5, n_max)
    f = [random.choice([0, 1]) for _ in range(n)]
    r_f = communication_complexity_rank(f)
    matrix = construct_matrix_representation(f, n)
    matrix = gaussian_elimination(matrix)
    order = compute_brauer_group_order(matrix)

    total_metric_value += order
    if n > n_max:
        n_max = n

mean_metric_value = total_metric_value / instances_tested
conjecture_holds = False
counterexample = ""

return {
    "metric_name": "Brauer Group Order",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9eee832c.py", line 91, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9eee832c.py", line 65, in run_trial
    matrix = gaussian_elimination(matrix)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9eee832c.py", line 38, in gaussian_elimination
    factor = 1 / matrix[i][i]
              ~^~~~~~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error before producing data that could confirm or falsify the conjecture. | next: Investigate and fix the division by zero error in the code, then rerun the test with a larger sample size to ensure robustness of the results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13429
2	propose	ollama_remote	glm4:latest	0	0	12346
3	preregistration	ollama_remote	glm4:latest	0	0	9960

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	8557
5	novelty	ollama_remote	glm4:latest	0	0	12135
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15918
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7846
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13876
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9045
10	judge	ollama_remote	glm4:latest	0	0	31173

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 134285 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/d7cf4659c191.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d7cf4659c191.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d7cf4659c191.tar.gz` (if generated)